Working code that does not explicitly need to be changed needs *not* to be changed. If it's working and the requirements are the same as they were, waking up the sleeping bear is far from your highest priority.

When working with legacy code, proceed incrementally, taking small steps that you can verify. Tweak the existing code as little as possible to get it to a state you can live with. Then you can ensure that the new code you write is as good—and as thoroughly tested—as possible. Making small improvements to existing code over time while writing new code as best you can will make the overall codebase better.

Getting Started with Legacy Code

When you're presented with a new codebase, your first job is to figure out exactly what the heck is going on. Toward that end, you should do three things immediately.

Get the Code in Source Control

These days it probably is already under source control, but you can't be too careful. Make sure you have all the access to any code repository that you need. While starting on a new legacy codebase is not the time to get fancy with new tools, you'll be much better served by using Git or some other source-control system that lets you easily create and manipulate branches. This will enable you to easily explore changes to the codebase using branches as scratch pads that can be kept or discarded as needed.

Get the Code Running

If the legacy project was conceived without much knowledge of Rails community practices (evidenced by the lack of tests), it won't be a surprise if the production environment is also a little sketchy. Conventional wisdom suggests that your development, staging, and production environments should be as similar as possible to prevent environment-specific errors. And although this is true in general, if your legacy project is being run by some random goofy server setup, it may be difficult or impossible to replicate that setup on your staging server, let alone your development machine.

If the exact production environment isn't available to you, the staging and development environments should be as generic as possible. If you can, push to migrate the production environment to a less fragile and more standard one as soon as possible. Container tools like Docker can be helpful here; it might be easiest to replicate the production environment in a Docker container rather than on your development computer.